

# CrocoLakeTools: A Python package to convert ocean observations to the parquet format

Enrico Milanese<sup>1</sup>, David Nicholson<sup>1</sup>, Gaël Forget<sup>2</sup>, and Susan Wijffels<sup>1</sup>

<sup>1</sup> Woods Hole Oceanographic Institution, United States of America <sup>2</sup> Massachusetts Institute of Technology, United States of America

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

## Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

## Summary

Investigations of the ocean state are possible thanks to the ever growing number of measurements performed with multiple instruments by different research missions. The vast and heterogeneous efforts have brought the community to define data storage conventions (e.g. CF-netCDF) and to assemble collections of datasets (e.g. the World Ocean Database). Yet, accessing these datasets often requires the usage of specialized tools, is generally inefficient over the cloud, and remains a major obstacle to new users in terms of pre-required knowledge and time. CrocoLakeTools is a Python package that addresses those challenges by providing workflows to convert various oceanographic datasets from their original format to a uniform parquet dataset with a shared schema.

## Statement of need

CrocoLakeTools is a Python package to build workflows that convert ocean observations from their original formats (e.g. netCDF, CSV) to parquet. CrocoLakeTools takes advantage of Python's well-established and growing ecosystem of open tools: it uses dask's parallel computing capabilities to convert multiple files at once and to handle larger-than-memory data. dask is already well-integrated with xarray and pandas, two widely used Python libraries for the treatment of array and tabular data, respectively, and with pyarrow, the API to the Apache Arrow library which is used to generate the parquet dataset.

Parquet is a data storage format for big tidy data which presents several advantages: it is language agnostic (it can be accessed with Python, Matlab, Julia and web development technologies); it offers faster reading performances than other tabular formats such as CSV; it is optimized for cloud systems storage and operations; it is widespread in the data science community, leading to a multitude of freely accessible tools and educational material.

CrocoLakeTools was developed with the goal of building and serving CrocoLake, a regularly refreshed database of sparse in-situ oceanographic observations that are pre-filtered to contain only quality-controlled measurements. CrocoLakeTools was designed to be used by researchers, engineers and data scientists in oceanography and to be accessed by the wider oceanographic community.

## State of the field

In-situ measurements of ocean characteristics are collected by a wide range of projects at different scales: from small research groups to international consortia, from time-limited regional explorations to continuous global observation arrays. This has led to a highly heterogeneous landscape of products that often differ for schema (variable names, data types), data format,

39 and access services. As a result, ingesting and analyzing data from different sources, e.g. for  
40 research purposes, presents technical and logistical challenges. The need for homogenized  
41 ocean data products has then prompted several efforts, in particular when it comes to sparse  
42 observations, which are characterized by discrete measurements that are generally irregular  
43 and sparse in time and space.

44 One of the most comprehensive projects to date is the World Ocean Database (WOD) ([Mishonov  
45 et al., 2024](#)), which contains more than 40 variables gathered from several sources (buoys,  
46 gliders, floats, bathythermographs, etc.). The earliest record dates back to 1772, the data  
47 is quality-controlled, and the database is regularly updated with the latest measurements  
48 available. The data is public and freely available through different interfaces ([WODselect](#),  
49 THREDDS, HTTPS, FTP) in ASCII, Comma Separated Value (CSV) and netCDF formats.  
50 WOD is maintained by the National Centers for Environmental Information (NCEI) of the  
51 United States' National Oceanic and Atmospheric Administration (NOAA).

52 Copernicus Marine Service is an initiative of the European Union that provides a wide range  
53 of ocean data products: physical, biogeochemical and sea ice data at regional and global  
54 scales, obtained from numerical models, in-situ observations and satellite observations. Among  
55 Copernicus' products, CORA([Szekely et al., 2019](#)) (Coriolis Ocean database for ReAnalysis)  
56 provides quality-controlled in-situ temperature and salinity measurements gathered from several  
57 global and regional networks.

58 The International Quality-controlled Ocean Database (IQuOD) ([Team, 2018](#)) is an effort by  
59 the oceanographic community “with the goal of producing the highest quality and complete  
60 single ocean profile repository along with (intelligent) metadata and assigned uncertainties”. It  
61 includes subsurface ocean profiles of several variables, paying particular attention to temperature  
62 measurements. The database is prepared by NCEI, and it is freely available in netCDF format  
63 through multiple channels (THREDDS, HTTPS, FTP).

64 The Ocean Data Platform by the non-profit HUB Ocean is among the youngest projects in the  
65 community, and it allows to find and access datasets from a catalog. The user can interact with  
66 the platform through different interfaces (SDK, REST API, OQS, JupyterHub workspaces),  
67 loading the datasets in tabular format.

68 Of particular interests to the community has been the data collected by the Argo ([Wong et  
69 al., 2020](#)) international program with a growing fleet of robotic floats, and several tools have  
70 been developed to handle such data. `argopy` ([Maze & Balem, 2020](#)) is a Python library that  
71 facilitates access and manipulation of data from the real-time observing system Argo ([Wong  
72 et al., 2020](#)). The user provides some filters (e.g. coordinate and time ranges, measurement  
73 name, etc) and `argopy` retrieves the relevant data from the Argo Global Data Assembly Center  
74 (GDAC). `ArgoData.jl`([Gael Forget, 2025](#)) and `OceanRobots.jl`([Gaël Forget & Gebbie, 2025](#)) are  
75 Julia packages that provide similar functionalities to `argopy`. `Argovis` ([Tucker et al., 2020](#)) is a  
76 REST API and web application hosted at the University of Colorado, Boulder (United States)  
77 and serving profile data in JSON format from the Argo program, and from ship and drifters  
78 missions. `oce` ([Kelley et al., 2022](#)) is a package written in R providing multiple functions to  
79 access oceanographic observations stored in a variety of formats. It focuses on datasets for  
80 physical oceanography with the goal of providing more tools in the future. `argoFloats` ([Kelley  
81 et al., 2021](#)) is a derived package that focuses on fetching and manipulating Argo data.

82 The above efforts often serve data in ASCII, CSV, JSON, or netCDF formats. For context,  
83 netCDF is a binary format that offers the advantage to be compact and efficient when dealing  
84 with multidimensional data, while the others have the advantage of being human-readable (but  
85 can be very inefficient for large datasets). None of these formats is optimized for cloud object  
86 storage, although there are ongoing efforts for netCDF (e.g. Zarr and Icechunk). For this  
87 reason, Parquet has been drawing more and more attention from the earth sciences community  
88 recently.

89 Parquet is a cloud-optimized binary format for tidy and large data (i.e. large tables) that is

language-agnostic. It is widely used in the data science and corporate worlds, and the software ecosystem around it is sound, mature, and growing. Table 1 presents an overview of the characteristics of Parquet and the other formats. We chose Parquet as the target format for CrocoLake because (1) it is optimized for cloud storage and cloud computing, (2) its mature software ecosystem includes packages in multiple coding languages to access it (Python, Julia, MATLAB, web technologies, etc.), and (3) novel users are generally more familiar with tabular data than with specialized oceanographic data formats.

As we aim to make CrocoLake easily accessible from a technical standpoint, the main drawbacks of Parquet at present are that (1) many workflows in ocean modeling are based on multidimensional data structures (not tabular ones) and (2) attaching attributes to the data is not straightforward (compared to netCDF). However, we see CrocoLake as a major building bloc in workflows that require fast access to sparse in-situ ocean observations, responding to necessities of data storage with fast access both on disk and the cloud, rather than as an end-all solution for ocean observations. For example, array-based storage formats might instead be more relevant to workflows that rely mostly on regular and gridded observations (such as satellite recordings).

Table 1. Comparison between different common file formats for oceanographic datasets.

| Feature Name              | CSV                       | netCDF                         | Parquet                               | ASCII            | JSON                      | Zarr + Icechunk                |
|---------------------------|---------------------------|--------------------------------|---------------------------------------|------------------|---------------------------|--------------------------------|
| Cloud-Optimized Structure | No<br>Tabular             | No<br>Array-based              | Yes<br>Tabular                        | No<br>Tabular    | No<br>Hierarchical        | Yes<br>Array-based             |
| Available tools in:       |                           |                                |                                       |                  |                           |                                |
| Python                    | Yes                       | Yes                            | Yes                                   | Yes              | Yes                       | Yes                            |
| Julia                     | Yes                       | Yes                            | Yes                                   | Yes              | Yes                       | Zarr only                      |
| MATLAB                    | Yes                       | Yes                            | Yes                                   | Yes              | Yes                       | Zarr only                      |
| Attributes descriptors    | Dedicated columns, header | Dictionary, accessed with data | Dictionary, separate access from data | Dedicated column | Dedicated field in object | Dictionary, accessed with data |

Another key feature of CrocoLakeTools is that it is fully open-source. Anyone can thus use CrocoLakeTools to build their own flavor of CrocoLake. We also welcome new contributors to CrocoLakeTools, for example by adding converters to support new datasets.

## Code architecture

### Converters

The core task of CrocoLakeTools is to take one or more files from a dataset and convert them to parquet, ensuring that CrocoLake's schema is followed. This is achieved through the methods contained in the Converter class and its subclasses. While the conversion of all datasets requires some general functionality (e.g. renaming the original variables to the final schema), each conversion requires specialized tools that differ between datasets (e.g. the map used to rename variables). CrocoLakeTools then contains the Converter class, which contains the methods shared across datasets, and from which converter subclasses inherit and implement the specific needs of each dataset.

## Workflow

### Local mirrors

The first step in our workflow is to retrieve original files from each data provider (Figure 1). The original source data follow the format, schema, nomenclature, and conventions defined by their provider (project, mission, scientist, etc.) independently of CrocoLake's workflow. Modules to download the original data are optional. They are expected to inherit from the Downloader class and be called `downloader<DatasetName>` (e.g. `downloaderArgoGDAC`). At the time of writing CrocoLakeTools is released with a downloader to build a local mirror of the Argo Global Data Assembly Center (GDAC), and we hope to support more data providers in the future. Whether a downloader module exists or the user downloads the data themselves, the original data is stored on disk and this is the starting point for the converter.

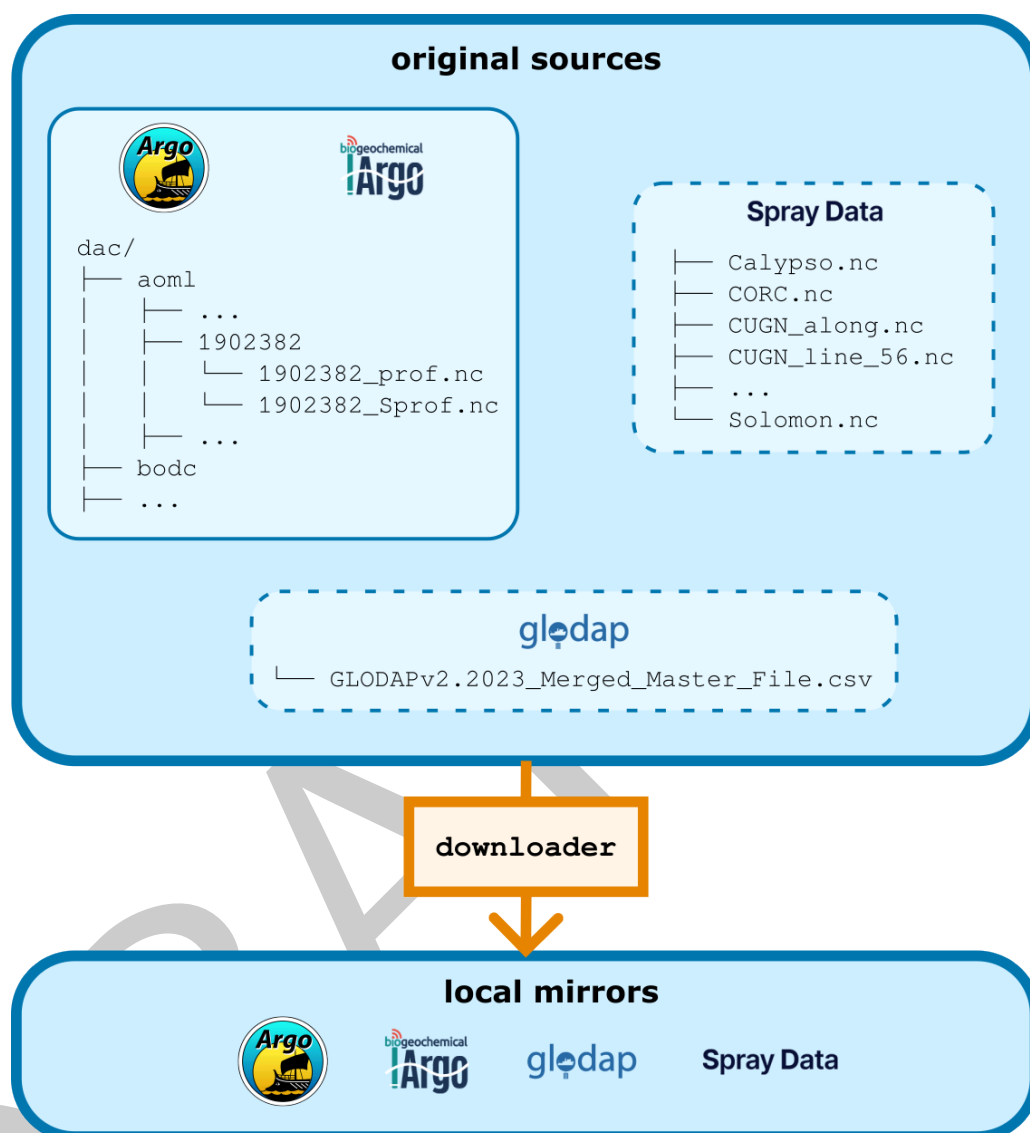
### Parquet datasets

The second step is to convert the data to parquet, and finally merge the datasets into CrocoLake (Figure 2). The core of CrocoLakeTools are the modules in the Converter class and its subclasses. Each original dataset has its own subclass called `converter<DatasetName>`, e.g. `converterGLODAP`; further specifiers can be added as necessary (e.g. at this time there are a few different converters for Argo data to prepare different datasets). The need for a dedicated converter for each project despite the usage of common data formats (e.g. netCDF, CSV) is due to differences in schema (e.g. variable names or units). Depending on the dataset, multiple converters can be applied. For example, to create CrocoLake, Argo data goes through two converters: 1. `converterArgoGDAC`, which converts the original Argo GDAC preserving most of its original conventions; 2. `converterArgoQC`, which takes the output of the previous step and applies some filtering based on Argo's QC flags and makes the data conforming to CrocoLake's schema. At this time CrocoLakeTools is released with converters for data from Argo (Wong et al., 2020), Spray Data (Sherman et al. (2002), Rudnick et al. (2016)) and GLODAP (Global Ocean Data Analysis Project, Key et al. (2015), Lauvset et al. (2016), Olsen et al. (2016)).

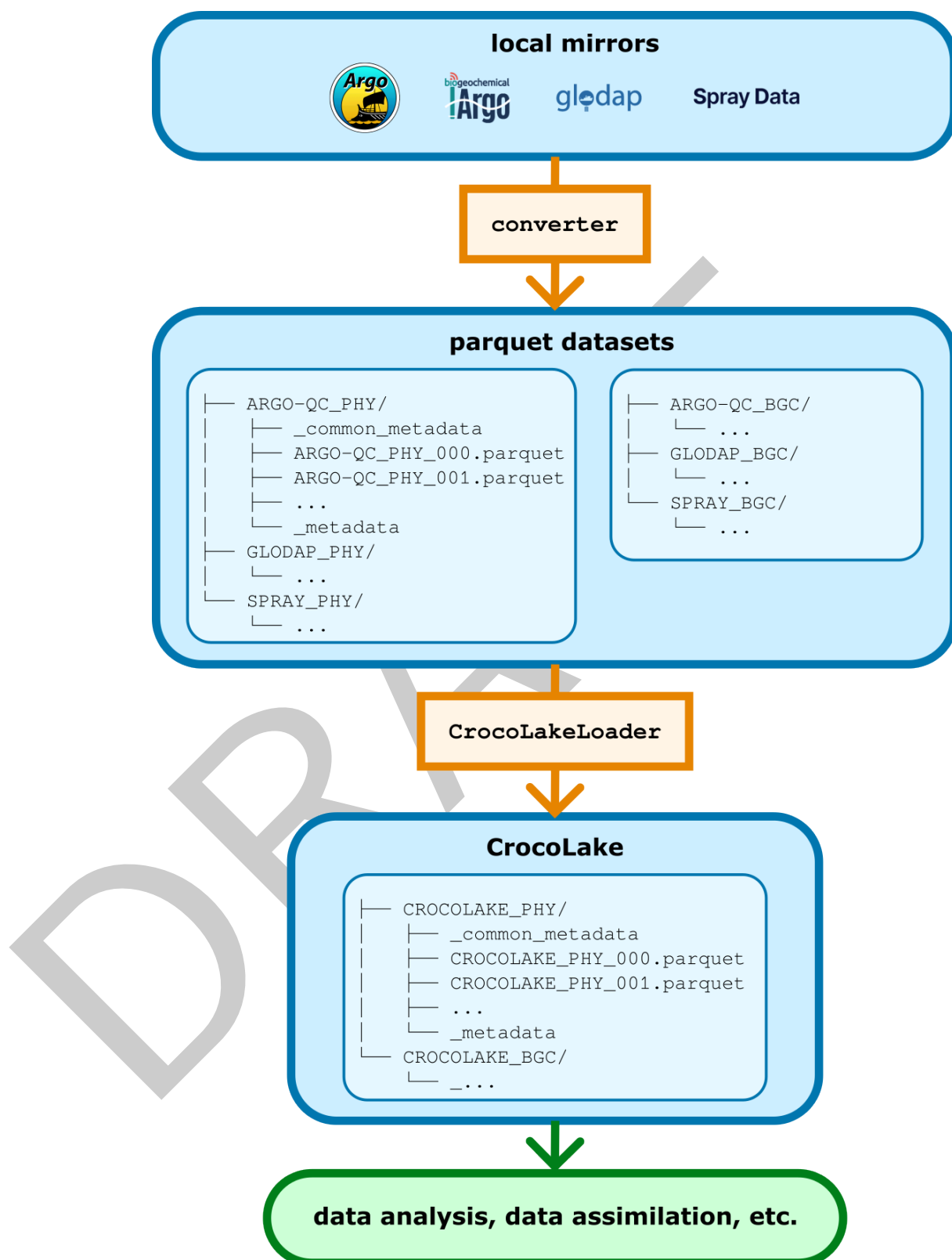
### CrocoLake

CrocoLake is one parquet dataset that contains all converted datasets merged together. This can be achieved with the script `merge_crocolake.py`. The script first creates a directory containing symbolic links to each converted dataset. It then uses the submodule `CrocoLakeLoader` to seamlessly load all the converted datasets into memory as one dask dataframe with a uniform schema, merges them into CrocoLake, and stores it back to disk.

CrocoLake can be accessed with several programming languages with just a few lines of codes: the submodules `CrocoLake-Python`, `CrocoLake-Matlab`, and `CrocoLake-Julia` contain tools and examples in some languages.



**Figure 1:** CrocoLake's workflow: 'downloader's. 'CrocoLakeTools' is set up to host modules that are dedicated to download the desired datasets from the web. It currently supports the download only of Argo data (solid line subset), and other datasets require the user to download them manually (dashed border subsets). Argo, Spray Data and GLODAP (Global Ocean Data Analysis Project) are the different data providers supported at the time of submission.



**Figure 2:** CrocoLake's workflow: 'converter's. 'converter's read the data in their original format, transform it following CrocoLake's conventions, converts it to parquet, and stores it back to disk. Each dataset is converted to its own parquet version. Thanks to the submodule 'CrocoLakeLoader', multiple parquet datasets are merged into a uniform dataframe which is saved to disk as CrocoLake. For each dataset, a version containing only physical variables ('PHY') or also biogeochemical variables ('BGC') can be generated.

## 155 Schema and metadata

156 The nomenclature, units and data types are generally based on Argo's (<https://vocab.nerc.ac.uk/collection/>)  
157 and are detailed in the online documentation. For CrocoLake's variables that are not present  
158 in the Argo program, we provide new names maintaining consistency with Argo's style.

159 The parquet format has two different types of metadata: metadata related to storage  
160 (e.g. schema, number of rows, etc.) and column attributes. We use the latter type to  
161 store information about the units for each variable present in CrocoLake. At the moment we  
162 are not including other metadata from the original sources (e.g. information about principal  
163 investigator, contact details, comments, etc.), but we are exploring solutions to include some  
164 of this in the future, based also on feedback from users.

## 165 Profile numbering

166 Ocean data is often accessed by profiles and we provide this functionality for CrocoLake too:  
167 the user can retrieve the profiles through the `CYCLE_NUMBER` variable, which is unique and  
168 progressive for each `PLATFORM_NUMBER` of each subdataset (`DB_NAME`). As each original product  
169 uses its own conventions, the `CYCLE_NUMBER` of some datasets is generated ad hoc during their  
170 conversion if no obvious match with `CYCLE_NUMBER` exists. The procedure for each dataset is  
171 detailed in the online documentation.

## 172 Quality control

173 CrocoLake contains only quality-controlled (QC) measurements. We rely exclusively on QCs  
174 performed by the data providers and at the time we do not perform any additional QC ourselves  
175 (although this might change in the future). Each parameter `<PARAM>` has a corresponding  
176 `<PARAM>_QC` flag that is generally set to 1 to indicate that the data is reliable. For Argo  
177 measurements, the original QC value is preserved, and only measurements with QC values of  
178 1, 2, 5, and 8 considered.

## 179 Measurement errors

180 Each parameter `<PARAM>` has a corresponding `<PARAM>_ERROR` that indicates a measurement's  
181 error as provided in the original dataset. When no error is provided, `<PARAM>_ERROR` is set to  
182 null.

## 183 Benchmarking

184 We here present a few examples of the advantages and disadvantages in accessing the generated  
185 parquet datasets. We compare access to Argo data in a few different scenarios between  
186 CrocoLake and `argopy` ([Maze & Balem, 2020](#)), which is widely used in the community. For  
187 the comparison, we use the parquet Argo data generated by `converterArgoGDAC` (i.e. without  
188 applying any filtering based on quality-control flags), as it is the version closest to the data  
189 served by `argopy`. The parquet dataset is stored on the cloud in a Amazon Web Services  
190 Simple Storage Solution (AWS S3) bucket, and `argopy` is called to access data from the Argo  
191 GDAC, so the data is always accessed over the internet. The parquet dataset is accessed with  
192 Python for comparison with `argopy`; we use `dask` but other packages exist to access parquet  
193 data (e.g. `duckDB`, `polars`) and the choice of package can affect the performance.

194 The goal of this benchmarking exercise is to provide a ballpark estimate of reading performances  
195 in a few different scenarios to showcase the potential of including oceanographic datasets in  
196 parquet format in scientific workflows; comparison with an existing tool as `argopy` is qualitative  
197 to provide a reference to help evaluate the performance of the parquet-based workflow. More  
198 generally, the reading performance of a given dataset depends on multiple factors beyond  
199 the dataset format itself (coding language, selection parameters, local vs cloud-based access,  
200 network connectivity, storage service, etc.).



Benchmarks were performed on a Lenovo ThinkPad P14s Gen 4 equipped with an AMD Ryzen 7 PRO 7840U processor (8 cores, 16 threads, up to 5.13 GHz) and 26 GiB DDR4 RAM. Network connectivity was provided via Wi-Fi: bandwidth measurements indicated an average download speed of 466.41 Mbit/s (10 runs, range: 363.37–555.70 Mbit/s) and an upload speed of 27.34 Mbit/s (10 runs, range: 26.98–27.48 Mbit/s); latency to google.com averaged 26.2 ms (10 measurements, range: 20.9–27.6 ms). All non-essential applications were closed during benchmarking, and measurements were collected immediately prior to the tests to ensure reproducibility.

The following Tables 2 and 3 summarize data loading times for PHY and BGC Argo datasets from AWS S3 (Parquet) versus the argopy Python library. Benchmarks include space and time filtering and loading by float WMO ID.

Table 2 – North West Atlantic Regional Subsets (2017)

| Data type | Subset size | Filter details                                                  | Parquet time | argopy time |
|-----------|-------------|-----------------------------------------------------------------|--------------|-------------|
| PHY       | Large       | Lat: [25,60], Lon: [-80,-30],<br>Date: 2017-01-01 to 2018-01-01 | 44.8 ± 0.4 s | 1266 ± 8.6s |
| PHY       | Small       | Lat: [55,60], Lon: [-50,-55],<br>Date: 2017-01-01 to 2018-01-01 | 19.5 ± 1.5 s | 63 ± 1.9s   |
| BGC       | Large       | Lat: [25,60], Lon: [-80,-30],<br>Date: 2017-01-01 to 2018-01-01 | 8.5 ± 0.7 s  | 954 ± 1.1s  |
| BGC       | Small       | Lat: [55,60], Lon: [-50,-55],<br>Date: 2017-01-01 to 2018-01-01 | 5.0 ± 2.3 s  | 108 ± 4.3s  |

Table 3 – Loading Full Float Records

| Data type | Floats    | Parquet time | argopy time   |
|-----------|-----------|--------------|---------------|
| PHY       | 1 float   | 4.2 ± 1.3 s  | 3.0 ± 0.3 s   |
| PHY       | 20 floats | 17.4 ± 0.8 s | 22.5 ± 1.1 s  |
| BGC       | 1 float   | 3.4 ± 1.8 s  | 6.2 ± 0.6 s   |
| BGC       | 20 floats | 8.0 ± 2.8 s  | 140.0 ± 6.0 s |

#### Notes:

- All timings are mean ± standard deviation measured over ten runs, except for the “Large” subsets in Table 2 which are measured over two runs.
- Large/small regional subsets (Table 2) differ by latitude/longitude bounding boxes (see above).
- The floats of the records in Table 3 were randomly picked and are identified by the following WMO IDs:
  - PHY, single float: 6902801;
  - PHY, 20 floats: 4902120, 6902707, 3901629, 6901177, 4901812, 4901765, 4901216, 4902322, 6902801, 6902753, 6902805, 5904749, 4901827, 4902419, 4902397, 4901755, 6902810, 5904007, 4902112, 1901596;
  - BGC, single float: 4902410;



228 – BGC, 20 floats: 6901754, 6901030, 4902409, 6902810, 6901750, 6902805, 6901182,  
229 6902686, 5904989, 6901603, 4902390, 5904770, 1901217, 6901751, 1901208,  
230 6901752, 6902812, 6901023, 6901758, 6901524.

## 231 Documentation and updates

232 The [documentation](#) describes the specifics of each dataset (e.g. what quality-control filter we  
233 apply to each dataset, the procedure to generate the profile numbers, etc.), and is updated  
234 every time a new feature is made available.

235 This document reflects the code at the time of publication. Any future updates and changes  
236 to the code will be documented in the code main branch, which will contain the most recent  
237 and best version available of CrocoLakeTools.

## 238 Citation

239 If you use CrocoLakeTools and/or CrocoLake, please do not limit yourself to citing this  
240 manuscript but also remember to cite the datasets that you have used as indicated in the  
241 documentation. For example, if your work relies on Argo measurements, acknowledge Argo  
242 ([Wong et al., 2020](#)). This is important both for the maintainers of each product to track their  
243 impact and to acknowledge their efforts that made your work possible.

## 244 Acknowledgements

245 This material is based upon work supported by the National Science Foundation under Grant  
246 No. 2311382, “Collaborative Research: Frameworks: A community platform for accelerating  
247 observationally-constrained regional oceanographic modeling.” G.F. acknowledges funding from  
248 NASA award 1686358, NOAA award 035724-00001, and ARIA’s POLEMIX award.

249 The Argo data are collected and made freely available by the International Argo Program and the  
250 national programs that contribute to it (<https://argo.ucsd.edu>, <https://www.ocean-ops.org>).  
251 The Argo Program is part of the Global Ocean Observing System.

## 252 References

- 253 Forget, Gael. (2025). *Euroargodev/ArgoData.jl* [Data set]. Zenodo. <https://doi.org/https://doi.org/10.5281/zenodo.3633717>
- 254
- 255 Forget, Gaël, & Gebbie, G. J. (2025). *JuliaOcean/OceanRobots.jl* [Data set]. Zenodo.  
256 <https://doi.org/https://doi.org/10.5281/zenodo.4718472>
- 257 Kelley, D. E., Harbin, J., & Richards, C. (2021). argoFloats: An r package for analyzing argo  
258 data. *Frontiers in Marine Science*, 8, 635922.
- 259 Kelley, D. E., Richards, C., & Layton, C. (2022). Oce: An r package for oceanographic analysis.  
260 *Journal of Open Source Software*, 7(71), 3594.
- 261 Key, R. M., Olsen, A., Heuven, S. van, Lauvset, S. K., Velo, A., Lin, X., Schirnack, C., Kozyr,  
262 A., Tanhua, T., Hoppema, M., & others. (2015). Global ocean data analysis project,  
263 version 2 (GLODAPv2). *Ornl/Cdiac-162, Ndp-093*.
- 264 Lauvset, S. K., Key, R. M., Olsen, A., Van Heuven, S., Velo, A., Lin, X., Schirnack, C., Kozyr,  
265 A., Tanhua, T., Hoppema, M., & others. (2016). A new global interior ocean mapped  
266 climatology: The 1× 1 GLODAP version 2. *Earth System Science Data*, 8(2), 325–340.
- 267 Maze, G., & Balem, K. (2020). Argopy: A python library for argo ocean data analysis. *Journal*  
268 *of Open Source Software*, 5.

- 269 Mishonov, A. V., Boyer, T. P., Baranova, O. K., Bouchard, C. N., Cross, S. L., Garcia, H. E.,  
270 Locarnini, R. A., Paver, C. R., Reagan, J. R., Wang, Z., & others. (2024). *World ocean*  
271 *database 2023*.
- 272 Olsen, A., Key, R. M., Van Heuven, S., Lauvset, S. K., Velo, A., Lin, X., Schirnick, C., Kozyr,  
273 A., Tanhua, T., Hoppema, M., & others. (2016). The global ocean data analysis project  
274 version 2 (GLODAPv2)—an internally consistent data product for the world ocean. *Earth*  
275 *System Science Data*, 8(2), 297–323.
- 276 Rudnick, D. L., Davis, R. E., & Sherman, J. T. (2016). Spray underwater glider operations.  
277 *Journal of Atmospheric and Oceanic Technology*, 33(6), 1113–1122.
- 278 Sherman, J., Davis, R. E., Owens, W., & Valdes, J. (2002). The autonomous underwater  
279 glider" spray". *IEEE Journal of Oceanic Engineering*, 26(4), 437–446.
- 280 Szekely, T., Gourrion, J., Pouliquen, S., Reverdin, G., & Merceur, F. (2019). *CORA, coriolis*  
281 *ocean dataset for reanalysis*.
- 282 Team, T. I. (2018). *International quality controlled ocean database (IQuOD) version 0.1*  
283 *- aggregated and community quality controlled ocean profile data 1772-2018 (NCEI*  
284 *accession 0170893)* [Data set]. NOAA National Centers for Environmental Information.  
285 <https://doi.org/https://doi.org/10.7289/v51r6nsf>
- 286 Tucker, T., Giglio, D., Scanderbeg, M., & Shen, S. S. (2020). Argovis: A web application for  
287 fast delivery, visualization, and analysis of argo data. *Journal of Atmospheric and Oceanic*  
288 *Technology*, 37(3), 401–416.
- 289 Wong, A. P., Wijffels, S. E., Riser, S. C., Pouliquen, S., Hosoda, S., Roemmich, D., Gilson,  
290 J., Johnson, G. C., Martini, K., Murphy, D. J., & others. (2020). Argo data 1999–2019:  
291 Two million temperature-salinity profiles and subsurface velocity observations from a global  
292 array of profiling floats. *Frontiers in Marine Science*, 7, 700.